

MEMÒRIA DE LA PRÀCTICA:

EXTRACCIÓ D'ENTITATS ANOMENADES

Uriel Gil Palma i Hugo Ortí Toledo

Introducció i Objectius de la Pràctica

El Reconeixement d'Entitats Anomenades (NER) és una tasca dins del processament del llenguatge natural que consisteix a identificar i classificar elements com noms de persones, organitzacions o llocs en un text. En aquest projecte utilitzem models de Camps Aleatoris Condicionals (CRF), una tècnica habitual per a problemes d'etiquetatge de seqüències com aquest. L'objectiu és construir un sistema capaç de processar informació lingüística de manera automàtica i amb un bon nivell de precisió.

Per entrenar i avaluar els models fem servir el corpus *CoNLL-2002*, i treballem amb les dades en espanyol i neerlandès. El treball es basa en les particions estàndard del corpus: el conjunt *train* s'utilitza per a l'entrenament, el conjunt *testa* per a la validació i ajust de paràmetres, i el conjunt *testb* per a l'avaluació final del rendiment dels models.

La pràctica consisteix a implementar un reconeixedor d'entitats amb la classe *nltk.tag.CRFTagger*. A més de la implementació bàsica, experimentem amb diferents combinacions de característiques (*features*), com ara informació morfològica o sufixos, per veure si milloren els resultats. També analitzem l'efecte de diversos esquemes d'etiquetatge, com BIO, BLOW o IO, amb l'objectiu de determinar quina opció funciona millor. Finalment, provem el model amb textos reals per valorar el seu comportament i la seva capacitat de predicció.

Enginyeria de Característiques - *Feature Extraction*

Per iniciar el desenvolupament del projecte vam decidir centrar primer el treball en la definició del mecanisme d'extracció de característiques, ja que aquest element constitueix la base sobre la qual es construeix tot el model i condiciona tant els experiments com l'anàlisi posterior dels resultats.

En aquest context, vam optar per dissenyar una funció pròpia basada en la nostra classe `FeatFunc_Class`, enlloc d'utilitzar la funció d'extracció per defecte del `CRFTagger` de NLTK. Aquesta elecció ens permet disposar de més control i flexibilitat a l'hora d'incorporar i avaluar diferents tipus de característiques. Mentre que la funció estàndard es limita a trets bàsics com la capitalització o la presència de dígit, la nostra aproximació ens facilita activar o desactivar característiques de manera sistemàtica, cosa que ens permet analitzar quines aporten més informació en cada cas i per a cada idioma.

Pel que fa al disseny, la classe l'hem implementat perquè sigui eficient i fàcil d'integrar amb els models. El mètode `__init__` actua com a configurador del model: emmagatzema les preferències (mitjançant valors booleans per activar o desactivar etiquetes i enters per definir el radi del context) i guarda recursos externs com les gasetes. Un aspecte clau d'aquesta fase és la pre-compilació d'expressions regulars, que evita processar-les repetidament per a cada paraula i optimitza la velocitat d'execució en comparar-les amb els tokens.

D'altra banda, el mètode especial `__call__` és el que permet utilitzar una instància com si fos una funció, permetent que el tagger de NLTK la cridi directament per a cada token, tal com requereix el paràmetre `feature_func`. Internament, el mètode consisteix en una sèrie de blocs condicionals que s'executen només si la característica corresponent ha estat activada. Dins d'aquests blocs s'apliquen funcions ràpides de Python (com `.islower()` o `.istitle()`) per a trets bàsics, mentre que per a trets més complexos, com la detecció de símbols o la generació de la "forma" de la paraula, s'utilitzen les expressions regulars compilades i mètodes de substitució. El resultat és una llista de *strings* que recullen els atributs seleccionats del token, oferint al model una representació bona i personalitzada.

Aquest disseny ens permet passar, en lloc d'una funció simple i estàtica, un objecte que manté tot el seu context, diccionaris i configuració de manera organitzada. Això facilita enormement l'experimentació, ja que es poden crear diferents instàncies de la classe amb configuracions variades (per exemple, una per a l'espanyol amb moltes característiques de context i una altra per al neerlandès més centrada en la morfologia) i comparar-ne el rendiment sense haver de modificar ni una sola línia del codi d'extracció.

A continuació es detallen el conjunt de característiques implementades, les quals hem seleccionat perquè considerem que poden ser discriminatives en tasques de Reconeixement d'Entitats Anomenades (NER):

- **Atributs de Paraula i Capitalització:** S'extreu la paraula base (en minúscules per evitar la dispersió del vocabulari) i s'analitza l'ús de majúscules (tot en minúscules, tot en majúscules o inici en majúscula). Això últim és un dels trets més forts per detectar noms propis en gairebé la majoria de llengües.
- **Informació Alfanumèrica:** Es detecta si el token és completament un conjunt de dígit, si conté números però no lletres o si és una combinació alfanumèrica. Això és clau per identificar entitats que inclouen codis, dates o referències tècniques.
- **Puntuació i Símbols:** Mitjançant les expressions regulars compilades a l'`__init__`, s'identifica la presència de signes de puntuació o símbols especials (com @, \$, €). Aquests

trets ajuden a descartar elements estructurals del text que rarament formen part d'una entitat i a classificar aquells que normalment en contenen (com "Covid-19").

- **Longitud de la Paraula:** Els tokens es classifiquen segons la seva llargada (1, 2, 3 i curta, mitjana, llarga). Certes entitats, com les sigles d'organitzacions, solen tenir longituds molt específiques i predictibles (*AI* → 2 i *UPC* → 3), la qual cosa aporta informació útil per a la classificació.
- **Forma General i Forma Curta:** Aquestes característiques converteixen la paraula en un patró abstracte (per exemple, "*Barcelona*" → "XXXXXXXX" → "Xx", "*06/07/2002*" → "ddpddpddd" → "dpdpd" i "*UPC*" → "XXX" → "X"). Això permet al model generalitzar i reconèixer l'estructura d'una entitat encara que la paraula concreta no hagi aparegut mai durant l'entrenament.
- **Afixos (Prefixos i Sufixos):** S'extreuen subcadenaes de 2 a 4 caràcters segons la longitud dels tokens. Moltes entitats comparteixen terminacions o inicis morfològics (com "-itat" o "Inter-") que poden donar pistes sobre la seva categoria.
- **Informació Morfosintàctica:** S'integren el POS-tag (categoria gramatical) i el lema. Saber que una paraula és un nom propi o un verb és essencial per decidir si pot ser una entitat. El lema ajuda a agrupar diferents flexions d'una mateixa paraula sota un únic tret, reduint la dispersió lèxica.
- **Context de Proximitat (Finestra mòbil):** El model no mira el token de forma aïllada, sinó que analitza una finestra de fins a dos tokens i dos POS-tags anteriors i posteriors. A més, s'inclouen bigrames de POS-tags per capturar relacions sintàctiques (per exemple, la seqüència "*Verb + Preposició*" i "*Preposició + Nom Propi*" solen indicar una localització).
- **Gasetes de Freqüència i Estàtiques:**
 - **Freqüència:** Creades dinàmicament des del corpus d'entrenament, identifiquen paraules que apareixen de forma recurrent com a entitats.
 - **Estàtiques:** Dictionaris externs de NLTK amb noms de països, ciutats o estats, que poden servir per millorar la precisió en entitats geogràfiques.

Aquesta estructura modular ens permet, en fases posteriors del projecte, realitzar una anàlisi de rellevància per provar diverses combinacions de característiques i quedar-nos només amb aquells trets que realment milloren el rendiment del model, evitant el sobreajustament (*fitting*).

Funcions Auxiliars

Per dur a terme tots els experiments i anàlisis prevists al projecte, vam veure necessari crear un conjunt de funcions auxiliars que ens ajudessin a evitar codi redundant i a mantenir el desenvolupament ordenat. Sense aquestes funcions, hauríem de repetir molts passos de preprocessament, càlcul i avaluació en cada experiment, cosa que faria el codi més repetitiu, propens a errors i molt més difícil de modificar.

Per això, hem implementat aquestes funcions auxiliars que cobreixen tot el flux de treball, des de la preparació de les dades fins a l'avaluació final del model. D'aquesta manera, es manté una estructura coherent amb la resta del projecte, es redueix la repetició de codi i es poden fer experiments comparatius de manera sistemàtica, modificant només els paràmetres d'interès (com l'idioma, l'esquema d'etiquetatge o les característiques) sense haver de tocar la resta del codi en cada nova prova.

Preparar Dades

La funció `preparar_dades` neix de la necessitat d'adaptar les dades provinents del corpus *CoNLL-2002* al format que espera el *CRFTagger* de NLTK, ja que el corpus tal com ve no és directament utilitzable pel model. A més, segons si volem entrenar el model o avaluar-lo, el format de sortida necessari és diferent. En lloc de tenir dues funcions separades que farien essencialment el mateix preprocessament, s'ha encapsulat tot en una sola funció controlada per un paràmetre booleà, evitant duplicació de codi i centralitzant tota la lògica de transformació en un sol lloc.

Hi ha també una altra raó clau per a l'existència d'aquesta funció: l'enriquiment de les dades amb lemes. El corpus *CoNLL-2002* no inclou informació de lematització, i afegir el lema de cada token com a característica addicional permet al model generalitzar millor, agrupant formes flexionades d'una mateixa paraula. Per obtenir aquests lemes usem *spaCy*, però de forma controlada: en lloc d'executar tot el *pipeline* sobre el text en brut, construïm el document manualment a partir dels tokens i les etiquetes POS originals del corpus. Això és important perquè si es deixés que *spaCy* analitzés el text des de zero, substituïria les etiquetes POS del corpus per les seves pròpies prediccions, trigant més i afegint possibles errors. Assignant manualment les etiquetes `tag_` a cada token, ens assegurem que el lematitzador treballi a partir de les etiquetes *CoNLL* originals.

La funció rep tres paràmetres: `corpus_brut`, la llista d'oracions en format *CoNLL*; `idioma`, que indica quin model de *spaCy* carregar ("es" per espanyol o "nl" per neerlandès); i `entrenament`, que determina el format de sortida. Internament, es carrega el model de *spaCy* corresponent i s'extreu únicament el component de lematització, descartant la resta del *pipeline*. Després itera oració per oració, construeix el Doc manualment, assigna les etiquetes POS originals i executa el lematitzador per obtenir el lema de cada token.

La sortida depèn del paràmetre `entrenament`. Si és `True`, cada token es representa com una tupla `((Token, POS, Lema), Etiqueta_NER)` agrupada per oració, que és exactament el format que espera `CRFTagger.train()`. Si és `False`, les dades i les etiquetes es separen en dues estructures paral·leles: `X` amb tuples `(Token, POS, Lema)` per fer prediccions, i `Y` amb les etiquetes NER reals per poder comparar-les amb les predites i calcular mètriques.

Generar Gasetes de Freqüència

La funció `generar_gasetes_frequencia` és una funció auxiliar que usem per construir les estructures de dades necessàries per a la característica de gasetes de freqüència, ja explicada anteriorment. La idea és que, per poder determinar si un token és un dels més freqüents com a entitat d'una categoria concreta, primer cal haver calculat aquest rànquing a partir del corpus d'entrenament. Aquesta funció és precisament la que genera aquest rànquing.

El motiu és aprofitar el coneixement estadístic del corpus d'entrenament: si una paraula apareix molt sovint etiquetada com a PER, LOC, ORG o MISC, és una evidència forta que en contextos nous probablement també ho sigui. Calcular aquestes freqüències de forma prèvia i emmagatzemar-les en un diccionari permet consultar-les de manera eficient durant l'extracció de característiques, sense haver de recórrer el corpus cada vegada.

La funció rep com a paràmetre `corpus_train`, el corpus d'entrenament en format *CoNLL* brut, és a dir, llistes de tuples `(Token, POS, NER)`. Internament, manté un comptador per a cada categoria d'entitat. Per a cada token del corpus, comprova si la seva etiqueta NER no és 0, i en cas afirmatiu, extreu el tipus d'entitat separant l'etiqueta pel guió, de manera que tant les etiquetes B-PER com I-PER

compten per a la mateixa categoria. El token es normalitza a minúscules abans de comptar-lo, per evitar que variants de capitalització es tractin com a paraules diferents.

Un cop comptades totes les freqüències, es construeix el rànquing: per a cada categoria, s'agafen les 1000 paraules més freqüents i se'ls assigna una posició ordinal, de manera que la més freqüent rep la posició 1, la segona la 2, i així successivament. La funció retorna un diccionari amb una entrada per a cada categoria (PER, LOC, ORG, MISC), on cada valor és un diccionari que mapeja tokens a la seva posició en el rànquing. Aquest diccionari és el que es passa posteriorment a la classe d'extracció de característiques per generar el tret de gaseta de freqüència.

Transformar Codificació

Les dades del corpus *CoNLL-2002* venen sempre en format BIO estàndard, però com hem comentat, volem comparar si altres codificacions d'etiquetatge permeten al model CRF obtenir millors resultats. Per fer-ho necessitem una funció que converteixi el corpus original a qualsevol de les altres codificacions abans d'entrenar el model. Aquesta és precisament la tasca de `transformar_codificacio`.

La funció rep dos paràmetres: `corpus_bio`, el corpus en format BIO original estructurat com una llista d'oracions on cada token és una tupla (`Token`, `POS-tag`, `Etiqueta_NER`); i `esquema`, una cadena de text que indica la codificació desitjada ("IO", "BIOW", "BIOE", "BIOEW" o "BIOW+"). Retorna un corpus amb la mateixa estructura però amb les etiquetes NER transformades.

La funció recorre el corpus oració a oració i token a token:

- Per a cada token, el primer que fa és comprovar si és un token fora d'entitat (O). En la majoria de codificacions simplement el manté com a O, però en el cas de BIOW+ li assigna una etiqueta del tipus O-ENT si és adjacent a una entitat: si el token anterior pertanyia a una entitat li assigna el tipus d'aquesta, i si no, comprova si el token següent és l'inici d'una entitat i li assigna el seu tipus.
- Per als tokens que sí pertanyen a una entitat, la funció extreu el prefix (B o I) i el tipus (PER, LOC, etc.), i comprova si el token és l'últim de la seva entitat mirant si el token següent és una I- del mateix tipus. Amb aquesta informació, aplica les regles de transformació de cada esquema:
 - Per a IO, simplement substitueix tots els prefixos per I-, eliminant qualsevol distinció d'inici.
 - Per a BIOW, detecta les entitats d'un sol token (aquelles que comencen amb B- i ahora són les últimes) i les converteix a W-, mantenint la resta igual.
 - Per a BIOE, afegeix E- únicament a l'últim token de les entitats compostes (les que acaben en I-), deixant les entitats d'un sol token com a B- sense modificar.
 - Per a BIOEW, combina les dues transformacions anteriors: les entitats d'un sol token es converteixen a W- i l'últim token de les entitats compostes passa a E-.
 - Finalment, BIOW+ aplica les mateixes regles que BIOW per als tokens d'entitat, però afegeix el comportament especial sobre els tokens O adjacents ja descrit.

Per observar el funcionament de la funció, considerem l'oració *"Mark Petersen is working at Google"*. La taula següent mostra com cada esquema transforma les etiquetes BIO originals per a aquesta oració:

	Mark	Petersen	is	working	at	Google.
BIO (Originals)	B-PER	I-PER	O	O	O	B-ORG
IO	I-PER	I-PER	O	O	O	I-ORG
BIOW	B-PER	I-PER	O	O	O	W-ORG
BIOE	B-PER	E-PER	O	O	O	B-ORG
BIOEW	B-PER	E-PER	O	O	O	W-ORG
BIOW+	B-PER	I-PER	O-PER	O	O-ORG	W-ORG

Agrupar Entitats

Abans de poder calcular cap mètrica, necessitem una manera de convertir una seqüència plana d'etiquetes NER en un conjunt d'entitats identificades. Per exemple, davant de la seqüència [B-PER, I-PER, O, B-LOC], el que volem obtenir és {(PER, 0, 1), (LOC, 3, 3)}, de manera que cada entitat representada com una tupla amb el seu tipus i les posicions d'inici i final dins la frase.

Sense aquesta conversió, **no podem comparar les prediccions del model amb les etiquetes reals** ni detectar encerts parcials. A més, com que estem treballant amb diverses codificacions d'etiquetatge diferents, la funció ha de ser capaç d'interpretar correctament la gramàtica de cadascun, ja que les regles de com s'obre i es tanca una entitat varien segons la codificació.

La funció recorre la seqüència d'etiquetes token a token mantenint un punter a l'entitat que s'està construint en cada moment (*entitat_actual*):

El primer que fa és ignorar qualsevol etiqueta O o O-ENT (pròpia del BIOW+), tancant l'entitat oberta si n'hi havia una. A partir d'aquí, el comportament varia segons la codificació:

- Per a l'esquema IO, com que no existeix cap marcador d'inici, una I- pot tant obrir una entitat nova com continuar-ne una d'existent. La funció distingeix els dos casos comprovant si ja hi ha una entitat oberta del mateix tipus: si és així, simplement n'amplia el límit final; si no, tanca l'anterior i n'obre una de nova.
- Per a la resta d'esquemes (BIO, BIOW, BIOE, BIOEW i BIOW+), la gramàtica és més estricta:
 - Una B- sempre obre una entitat nova, tancant l'anterior si cal.
 - Una W- (present als esquemes que la incorporen) representa una entitat d'un sol token que neix i mor immediatament sense deixar cap entitat oberta.
 - Una I- només pot allargar una entitat si ja n'hi ha una d'oberta del mateix tipus. Si apareix sola (òrfena), simplement s'ignora: no es genera cap entitat en aquella posició, cosa que implícitament penalitza el model ja que perd un possible encert en el càlcul del F1.
 - Finalment, una E- tanca l'entitat oberta si és del mateix tipus, i si apareix òrfena també s'ignora.

Al final del recorregut, si queda alguna entitat oberta (és a dir, una seqüència que arriba al final de la frase sense haver trobat cap marcador de tancament), es tanca i s'afegeix al resultat. La funció retorna un conjunt (*set*) de tuples, cosa que elimina automàticament qualsevol duplicat i facilita les operacions d'intersecció i diferència que es fan posteriorment a *f1_score_parcial*.

Mètrica F1

Un cop tenim les entitats agrupades, necessitem una manera de mesurar com de bé ho ha fet el model. Tot i que l'*Accuracy* és la mètrica més habitual en classificació, en NER resulta poc informativa i enganyosa: la gran majoria dels tokens d'un corpus estan etiquetats com a O (fora d'entitat), de manera que un model que simplement predís O per a tots els tokens obtindria un *Accuracy* molt alt sense haver detectat cap entitat. El F1-Score, en canvi, se centra exclusivament en les entitats i mesura l'equilibri entre les que el model detecta correctament i les que es deixa o prediu malament, donant una imatge molt més fidel del rendiment real del model.

A més, en lloc d'usar un F1 binari estricte (on o encertes completament o no comptes res), hem decidit implementar **una variant parcial** que recompensa també les deteccions incompletes. Això té sentit en NER: si el model detecta "*Ministerio de Sanidad y Consumo*" i prediu únicament "*Ministerio de Sanidad*", ha fet una feina parcialment correcta que mereix ser reconeguda.

La funció rep tres paràmetres: `y_true`, la llista d'etiquetes reals del corpus; `y_pred`, la llista d'etiquetes predites pel model; i `esquema`, el nom de la codificació utilitzada (com "BIO" o "BIOEW"), que es passa directament a `agrupar_entitats` per garantir que les entitats s'interpreten segons la gramàtica correcta.

La funció recorre totes les frases del corpus i per a cada frase obté les entitats reals i predites cridant `agrupar_entitats`. Acumula quatre comptadors globals: el total d'entitats reals, el total de prediccions, els encerts exactes i els encerts parcials:

- **Els encerts exactes (correctes)** s'obtenen directament com la intersecció dels dos conjunts d'entitats. Dues entitats coincideixen exactament si tenen el mateix tipus i exactament els mateixos índexs d'inici i final, és a dir, **la tupla (tipus, inici, fi) és idèntica**. Aquestes entitats es resten dels dos conjunts per no tornar-les a comptar en el pas següent.
- **Els encerts parcials** es calculen sobre les entitats restants mitjançant un doble bucle. Per a cada entitat predita no encertada exactament, es busca si hi ha alguna entitat real del mateix tipus amb la qual es solapi posicionalment. La condició de solapament és:

$$\text{inici}_{\text{real}} \leq \text{fi}_{\text{pred}} \quad \text{i} \quad \text{inici}_{\text{pred}} \leq \text{fi}_{\text{real}}$$

Per evitar que una mateixa entitat real es compti dues vegades, es manté un conjunt `reals_emparellades` que registra quines ja han estat aparellades. En trobar un solapament, s'incrementa el comptador de parcials i es passa a la següent predicció.

Un cop acumulats tots els comptadors, es calculen la Precisió i el Recall, on els encerts parcials valen la meitat d'un encert exacte:

$$\text{Precisió} = \frac{\text{Correctes} + 0.5 \times \text{Parcials}}{\text{Total Prediccions}} \qquad \text{Recall} = \frac{\text{Correctes} + 0.5 \times \text{Parcials}}{\text{Total Reals}}$$

Finalment, el F1-Score es calcula com:

$$F_1 = \frac{2 \times \text{Precisió} \times \text{Recall}}{\text{Precisió} + \text{Recall}}$$

La funció retorna les tres mètriques (F1, Precisió, Recall), i en el cas que no hi hagi cap predicció ni entitat real retorna (0, 0, 0) per evitar divisions per zero.

Preparar Text Real

La funció `preparar_text_real` existeix per cobrir un cas d'ús diferent al de `preparar_dades`: mentre que aquesta última treballa amb dades ja preprocessades del corpus *CoNLL-2002*, `preparar_text_real` s'encarrega de processar text completament en brut, com ara text extret d'internet o qualsevol altra font externa. Aquest text no ve tokenitzat ni té etiquetes POS ni lemes, de manera que cal aplicar tot el pipeline d'anàlisi lingüística des de zero. A més, el resultat d'aquest pipeline **ha de tenir exactament la mateixa estructura i codificació que les dades d'entrenament**, ja que el CRF només pot fer prediccions útils si les features extretes coincideixen amb les que ha vist durant l'entrenament. Si el POS-tag vingués en un format diferent (per exemple NOUN en lloc de NC), la feature `POSTAG_NOUN` no coincidiria amb cap de les features apreses (`POSTAG_NC`), i el model perdria completament aquesta informació.

Per fer-ho hem utilitzat *Stanza*, que permet obtenir la tokenització, l'etiquetatge morfosintàctic i la lematització amb etiquetes prou properes als estàndards del corpus *CoNLL-2002*. NLTK es va descartar per no tenir suport robust per a l'espanyol ni el neerlandès, i *spaCy* perquè utilitza una codificació molt diferent del format del corpus d'entrenament.

La funció rep tres paràmetres: una llista de textos en brut, l'identificador de l'idioma ("es" o "nl") i un diccionari de traducció (*mapping*) per al neerlandès. Internament, instancia el pipeline de *Stanza* amb els processadors de tokenització, POS i lematització, i extreu el text, el lema i el POS-tag de cada paraula. És en aquest punt on s'aplica una capa de normalització estricta que depèn de l'idioma:

- **Espanyol:** *Stanza* retorna l'atribut `xpos`, que es similar a la del corpus però amb més nivell de detall (com VMIS3S0 per a alguns verbs). Cal aplicar el trucament dels postags per obtenir la mateixa codificació que utilitza el corpus:
 - Els verbs es redueixen als tres primers caràcters (VMIS3S0 → VMI).
 - Categories com dates o xifres (W, Z, I) a un sol caràcter.
 - Els signes de puntuació (que comencen per F) es capitalitzen de forma específica per preservar distincions com Fp o Fc.
 - La resta de categories es trunquen als dos primers caràcters.
- **Neerlandès:** *Stanza* retorna etiquetes `upos`, que no coincideixen amb la codificació pròpia del *CoNLL-2002* neerlandès. Per resoldre-ho hem usat el diccionari auxiliar `mapeig_nl`, que tradueix cadascuna de les etiquetes universals generades per *Stanza* al format esperat pel model (per exemple, NOUN → N, VERB → V, ADJ → Adj).

Anotació: Aquestes regles de normalització s'han determinat simplement comparant les etiquetes del corpus *CoNLL-2002* amb les que generava *Stanza* sobre les mateixes oracions, observant token a token on varien i ajustant la transformació fins a obtenir el format correcte.

Un cop normalitzades les etiquetes, la funció retorna una llista de llistes on cada token s'empaqueta com una tupla (`Token`, `POS`, `Lema`), el format exacte que espera la classe `FeatFunc_Class` per extreure les característiques i que garanteix la compatibilitat completa amb el model entrenat.

Per verificar la qualitat de la normalització, hem comparat les etiquetes POS originals del corpus *CoNLL-2002* amb les generades per la nostra funció sobre les mateixes oracions:

Token	CoNLL-2002	Stanza
Esta	DD	DD

Token	CoNLL-2002	Stanza
Vandaag	Adv	Adv

página	NC	NC
web	AQ	NC
lleva	VMI	VMI
un	DI	DI
(...)	(...)	(...)

is	V	V
Floralux	N	N
dus	Adv	Adv
met	Prep	Prep
(...)	(...)	(...)

Les diferències observades són molt puntuals i, en la majoria dels casos, no es deuen a errors de normalització sinó a etiquetes qüestionables en el propi corpus. Per exemple, "web" apareix etiquetat com a AQ (adjectiu qualificatiu) quan podria argumentar-se que és un NC (nom comú), una distinció que depèn de la interpretació gramatical adoptada.

Predir Text Cru

La funció `predir_text_cru` és la funció d'interfície final que connecta totes les parts del codi: rep text completament en brut, el processa, fa les prediccions i presenta els resultats de manera llegible. És, en certa manera, la conclusió del projecte, on veiem la utilitat i rendiment del model en un entorn real fora del corpus.

Aquesta funció encapsula tot el flux de predicció sobre text real en una sola crida. Sense aquesta funció, s'hauria de cridar manualment `preparar_text_real`, passar el resultat al tagger i iterar sobre les prediccions per interpretar-les. `predir_text_cru` ho fa tot de forma seqüencial i transparent, i a més filtra i mostra únicament les entitats detectades, descartant els tokens etiquetats com a O, que no són entitats.

La funció rep quatre paràmetres: `textos`, la llista de strings a analitzar; `tagger`, l'objecte *CRFTagger* ja entrenat; `idioma`, que es passa directament a `preparar_text_real`; i `mapeig_neer`, el diccionari de traducció POS per al neerlandès que s'ha comentat abans, que és opcional i per defecte s'inicialitza com un diccionari buit si no es proporciona.

Internament, el flux és directe: primer crida `preparar_text_real` per obtenir les oracions en format (`Token`, `POS`, `Lema`), tal com s'ha explicat anteriorment, i tot seguit crida `tagger.tag_sents` sobre aquestes dades per obtenir les prediccions. Després itera sobre cada oració predita i imprimeix el text original seguit de les tuples dels tokens que han estat etiquetats com a entitats, indicant per a cadascun l'etiqueta assignada. Si cap token d'una oració és una entitat, s'indica explícitament.

La funció retorna `prediccions_cru`, una llista d'oracions on cada oració és una llista de tuples (`Token`, `Etiqueta_NER`) amb les prediccions del model, el mateix format de sortida que produeix `tagger.tag_sents`, tot i que mai s'arriba a usar aquesta llista; els *prints* ja són suficients.

Cerca Greedy

L'objectiu d'aquesta fase és determinar quines característiques (*features*) aporten valor real al model CRF i quines són redundants. En lloc d'entrenar un model per a cada combinació possible de features (el que seria un *Grid Search* exhaustiu amb un cost computacional molt alt i moltes combinacions), hem optat per una cerca *Greedy*, que construeix la configuració òptima de manera incremental i eficient.

La lògica és la següent:

1. Es parteix d'un model base completament buit, amb totes les *features* desactivades; inicialment no s'usa cap.
2. En cada iteració s'avaluen totes les *features* candidates de manera independent. La que aconsegueix el millor F1-Score Parcial sobre el conjunt de validació s'incorpora permanentment a la configuració.
3. El procés es repeteix fins que cap feature candidata aconsegueix millorar el millor F1 acumulat o fins que no hi quedi cap d'inactiva. En aquest punt, l'algorisme s'atura de manera natural.

La separació entre conjunt d'entrenament i conjunt de validació és fonamental en aquest procés: en cada iteració, el model **s'entrena exclusivament amb les dades d'entrenament**, però la decisió de quina feature és la guanyadora es pren mirant els **resultats sobre el conjunt de validació**. D'aquesta manera ens assegurem que la selecció de característiques reflecteix la capacitat del model de generalitzar a dades que no ha vist mai, i no una simple memorització del corpus d'entrenament. Com a conseqüència, l'algorisme descarta de forma natural qualsevol característica que, tot i millorar el rendiment sobre les dades d'entrenament, no generalitzi bé a dades noves, prevenint així el sobreajustament (*overfitting*) sense necessitat d'aplicar cap mecanisme explícit.

Les *features candidates* es divideixen en dos grups. D'una banda, les features booleanes, que simplement s'activen o es desactiven (True/False). De l'altra, les features de context (*usa_context_paraula* i *usa_context_postag*), que poden prendre els valors 0, 1 o 2, on 0 significa que no s'usa context, 1 que es mira el token immediat anterior i posterior, i 2 que s'amplia la finestra a dos tokens en cada direcció.

Per gestionar ambdós tipus de manera uniforme dins del mateix bucle, les opcions de context s'han codificat com a cadenes del tipus '*usa_context_paraula=1*' i '*usa_context_paraula=2*', tractant cada valor com un candidat independent. La funció auxiliar *aplicar_feature* s'encarrega d'interpretar aquestes cadenes i assignar el valor enter corresponent a la configuració. Això té una implicació important: si per exemple *usa_context_paraula=1* guanya una iteració i s'incorpora a la configuració, l'opció *=2* es descarta automàticament de la llista de candidates, ja que no té sentit provar un valor diferent per a una feature que ja ha estat fixada.

Paral·lisme

Donat que l'execució seqüencial del *Greedy* resultava també molt lenta, ja que cada iteració requeria entrenar i avaluar tants models com *features* restants, un darrere l'altre, vam identificar que les avaluacions dins d'una mateixa iteració són completament independents entre si: cada *feature* es prova de manera aïllada sobre la mateixa configuració base, sense cap dependència amb les altres. Això fa que el bucle intern sigui paral·litzable, on cada core del processador pot avaluar una *feature* diferent simultàniament.

Per implementar-ho hem utilitzat la llibreria *joblib*, que permet distribuir feina entre els cores disponibles de manera senzilla. El nucli d'aquest paral·lisme és la funció *avaluar_una*, que encapsula tot el procés d'avaluació per a una única *feature* candidata.

Funció d'avaluació (*avaluar_una*)

És una funció que cada core executa per separat. La seva lògica és la següent:

1. Es copia la configuració actual amb *deepcopy* per evitar interferències entre processos.
2. S'activa la feature corresponent i es crea una instància de *FeatFunc_Class* amb aquesta configuració.

3. S'entrena un *CRFTagger* sobre el conjunt d'entrenament i es realitza les prediccions sobre el conjunt de validació.
4. Per últim, es calcula el F1-Score Parcial.

Cada core escriu el seu model en un fitxer propi per evitar col·lisions entre processos que s'executen simultàniament. La funció retorna una tupla amb el F1 obtingut, el nom de la feature i el registre complet de la configuració.

El bucle principal de la cerca *Greedy* coordina tot el procés iteració a iteració. Al principi de cada ronda, `Parallel(n_jobs=-1)` llança `avaluar_una` per a totes les *features* restants (candidates) simultàniament, distribuint-les entre tots els cores disponibles. Els cores treballen en paral·lel i, quan tots han acabat, `joblib` acumula els resultats en una llista `resultats_ronda` que el procés principal recull de cop.

A partir d'aquí, s'itera sobre aquesta llista per identificar quina *feature* ha obtingut el millor F1, s'incorpora a la configuració global i es retira de la llista de candidates. Si la *feature* guanyadora era una opció de context (per exemple `usa_context_paraula=1`), també s'elimina l'alternativa (`=2`) per evitar inconsistències i optimitzar el procés. Si cap feature millora el F1 acumulat, l'algorisme s'atura.

Finalment, tots els registres acumulats s'exporten a un CSV per analitzar l'evolució del rendiment.

Anàlisi dels resultats

En aquest apartat analitzarem i interpretarem els resultats obtinguts durant el projecte. Més enllà de les mètriques, examinarem quines característiques va seleccionar el *Greedy Search* i per què tenen sentit lingüísticament, compararem els diferents esquemes d'etiquetatge (BIO, BIOE, BIOEW, etc.) i el seu efecte sobre el CRF, i avaluarem com es comporten els models finals sobre text cru. Tot plegat ens ajudarà a entendre com les decisions de disseny afecten directament la detecció d'entitats, i també veurem com de diferents són l'espanyol i el neerlandès des d'un punt de vista morfosintàctic.

Cerca de Característiques (*Greedy*)

Tal com hem comentat, el mètode *Greedy* parteix d'un model sense cap feature activa i, iteració a iteració, incorpora la característica que més millora l'F1-Score Parcial del conjunt de validació.

A continuació veurem com evoluciona el rendiment del model a mesura que es van afegint característiques:

Evolució de la cerca *Greedy* per a l'Espanyol:

Iteració	Característica seleccionada	F1-Score	Millora
1	<code>usa_forma</code>	0.4896	0.4896
2	<code>usa_afixos</code>	0.7044	0.2148
3	<code>usa_context_paraula = 2</code>	0.7571	0.0527
4	<code>usa_paraula_base</code>	0.7677	0.0106
5	<code>usa_context_postag = 1</code>	0.7732	0.0055
6	<code>usa_postag</code>	0.7770	0.0038
7	<code>usa_lema</code>	0.7788	0.0018
8	<code>usa_puntuacio</code>	0.7805	0.0017

9	usa_es_minusc	0.7807	0.0002
10	usa_gasetes	0.7824	0.0017
11	usa_forma_curta	0.7852	0.0028
12	usa_es_majusc	0.7862	0.0010
13	Cap feature millora l'F1		

El guany més gran el veiem amb `usa_forma`, que per si sola porta el model a un F1 de gairebé 0.49. Convertir cada paraula en un patró abstracte (com *"Madrid"* → *"Xxxxx"*) permet al CRF **detectar entitats a partir de la forma visual de les paraules** sense necessitat de conèixer el significat de cap. Just després, `usa_afixos` suma gairebé 0.22 al F1-Score: reconèixer prefixos i sufixos permet al model tant descartar falsos positius, terminacions com *"-ando"*, *"-mente"* o *"-aba"* rarament pertanyen a una entitat, com reforçar deteccions correctes, ja que certs prefixos com *"Inter-"* o *"Euro-"* solen aparèixer en noms d'organitzacions.

Un cop el model té clara la morfologia, necessita mirar al seu voltant. Afegir el **context de paraules** (`usa_context_paraula=2`) dona el tercer salt més gran, cosa que té sentit: les entitats gairebé mai apareixen soles, i les preposicions o verbs del voltant solen ser pistes molt útils. Més endavant, `usa_paraula_base`, `usa_postag` i `usa_lemma` afinen les prediccions aportant informació semàntica i morfològica concreta. Conèixer la paraula exacta en minúscules permet al model memoritzar entitats freqüents del corpus, el POS-tag confirma si la paraula és un nom propi o un verb, i el lema agrupa flexions d'una mateixa paraula sota una única representació, reduint la dispersió lèxica i ajudant a generalitzar millor.

A partir de la iteració 8, les millores són molt petites. `usa_puntuacio` i `usa_es_minusc` aporten dècimes a l'F1 Score, ajudant a descartar tokens que no poden ser entitats. `usa_forma_curta` i `usa_es_majusc` entren a les iteracions 11 i 12 amb guanys igualment molt petits, acabant de polir casos molt concrets on la forma abstracta curta o la detecció de paraules en majúscules aporten algun encert addicional.

Val la pena fixar-se en quan apareixen les gasetes (`usa_gasetes`), a la iteració 10, amb una millora de només 0.0017 punts. Que els diccionaris externs entrin tan tard i aportin tan poc és en realitat un bon senyal: el model ja ha après a identificar entitats per patrons, i les gasetes simplement corregeixen alguns casos molt puntuals.

L'algorisme s'atura a la iteració 13, quan cap feature restant millora el F1 acumulat. Això és positiu perquè evita afegir característiques que només aportarien soroll, mantenint el model més lleuger i generalitzable.

Evolució de la cerca Greedy per al Neerlandès:

Iteració	Característica seleccionada	F1-Score	Millora
1	usa_afixos	0.5205	0.5205
2	usa_forma_curta	0.6600	0.1395
3	usa_context_paraula = 1	0.7212	0.0612
4	usa_paraula_base	0.7373	0.0161
5	usa_context_postag = 2	0.7526	0.0153
6	usa_longitud	0.7597	0.0071

7	usa_postag	0.7666	0.0069
8	usa_lema	0.7672	0.0006
9	usa_forma	0.7717	0.0045
10	usa_es_digit	0.7718	0.0001
11	usa_simbol	0.7719	0.0001
12	Cap feature millora l'F1		

L'anàlisi del *Greedy* per al neerlandès mostra uns resultats molt similars en termes d'estructura, però amb diferències lingüístiques prou interessants que val la pena comentar.

El guany inicial més gran el veiem amb *usa_afixos*, que aporta més de 0.52 punts d'F1 a la primera iteració. Té sentit: el neerlandès fa un **ús molt intensiu de sufixos i prefixos** específics per a noms de llocs, professions i entitats (terminacions com *“-burg”, “-dam”* o *“-straat”*), de manera que reconèixer l'estructura interna de la paraula ja resol la majoria dels casos. Just després, *usa_forma_curta* suma 0.14 més, i combinats, els dos permeten al model arribar a un F1 de 0.66 simplement mirant la "silueta" de cada paraula.

Pel que fa al context, el model segueix una estratègia diferent a la de l'espanyol. Per al context de paraules exactes, amb mirar el token immediat anterior i posterior (*usa_context_paraula=1*) n'hi ha prou, ampliar la finestra no aportava res. En canvi, per a l'estructura gramatical sí que necessita una finestra més àmplia (*usa_context_postag=2*), cosa que indica que en neerlandès la posició sintàctica de l'entitat dins la frase és més determinant que el vocabulari específic del voltant.

A les iteracions següents, *usa_paraula_base*, *usa_postag* i *usa_lema* afinen les prediccions amb guanys petits però útils. Destaca especialment *usa_longitud*, una característica que l'espanyol no va arribar a seleccionar: com que el neerlandès forma compostos molt llargs ajuntant diverses paraules, la longitud del token resulta ser un indicador útil per detectar entitats. Finalment, *usa_forma* entra a la iteració 9 amb un guany petit (0.0045), molt lluny del paper protagonista que tenia en l'espanyol, i les dues últimes features seleccionades, *usa_es_digit* i *usa_simbol*, aporten millores gairebé inapreciables (0.0001 cadascuna), servint únicament per acabar de filtrar tokens numèrics o amb símbols que puguin generar confusió.

Crida l'atenció l'absència de *usa_gasetes*, que creiem que segurament l'algorisme ha descartat per dues raons. D'una banda, els diccionaris externs de NLTK tenen un fort biaix cap a l'anglès, i aplicats al neerlandès introduïen més soroll que encerts. D'altra banda, les gasetes de freqüència extretes del corpus tampoc han aportat res, cosa que indica que el model ja havia après a detectar entitats prou bé a través de patrons morfològics i contextuals, i memoritzar vocabulari específic de l'entrenament no millorava els resultats sobre validació.

És interessant comparar directament el rol de *usa_forma* en els dos idiomes. En espanyol és la característica més important de totes, aportant quasi 0.49 punts d'F1 a la primera iteració, mentre que en neerlandès no apareix fins a la iteració 9 amb una millora de tan sols 0.0045. La raó és lingüística: **en espanyol les entitats tendeixen a seguir patrons ortogràfics molt regulars** (una paraula amb majúscula inicial, d'una longitud moderada), de manera que el patró abstracte "Xxxxx" és molt discriminatiu. En neerlandès, en canvi, **les paraules compostes molt llargues generen patrons únics i poc reutilitzables**, cosa que fa que la forma abstracta perdi molt del seu valor. Una cosa similar passa amb *usa_longitud*: l'espanyol la descarta perquè la longitud no discrimina prou bé entre entitats i no-entitats, mentre que en neerlandès resulta útil precisament perquè els compostos formats per diverses paraules juntes solen ser notablement més llargs que les paraules comunes.

L'algorisme s'atura a la iteració 12, quan cap feature restant millora el F1 de 0.7719 assolit a la iteració anterior, evitant així afegir soroll i mantenint un model robust i generalitzable.

Comparativa entre Idiomes

Un cop analitzats els dos processos de cerca, podem comparar directament els resultats obtinguts:

	Espanyol	Neerlandès
F1-Score final	0.7862	0.7719
Iteracions fins a l'aturada	13	12
Features seleccionades	12	11

L'espanyol obté un F1 lleugerament superior sobre el conjunt de validació. Tots dos models coincideixen en algunes features (morfologia superficial, lema, afixos, paraula base i context sintàctic), però com hem vist, el pes i l'ordre en què s'incorporen varia segons l'estructura de cada llengua.

Comparativa dels Esquemes de Codificacions

Per avaluar l'impacte dels diferents esquemes d'etiquetatge, vam mantenir les features òptimes obtingudes al *Greedy Search* per a cada idioma i vam canviar únicament com es codifiquen les etiquetes del corpus.

L'esquema d'etiquetatge defineix com el model aprèn els límits de les entitats. Vam provar sis variants: el BIO estàndard (Begin, Inside, Outside), la simplificació IO (sense indicador d'inici), les extensions BLOW, BIOE i BIOEW (que afegeixen marcadors per a entitats d'una sola paraula i finals d'entitat), i l'esquema BLOW+, que enriqueix els tokens O adjacents a una entitat amb informació del seu tipus.

A continuació es presenten els resultats obtinguts sobre el conjunt de validació:

1. Anàlisi per a l'Espanyol

Esquema (codificació)	F1-Score	Precisió	Recall
BIO	0.7862	0.7862	0.7691
IO	0.7798	0.7992	0.7613
BLOW	0.7831	0.8007	0.7661
BIOE	0.7889	0.8069	0.7717
BIOEW	0.7843	0.8018	0.7675
BLOW+	0.7633	0.7790	0.7482

Els resultats per a l'espanyol mostren que afegir complexitat a la representació de les etiquetes pot ajudar, sempre que s'ajusti a les característiques de l'idioma.

El millor resultat l'obté BIOE, l'únic esquema que supera el BIO base en F1. L'espanyol genera sovint entitats llargues formades per múltiples paraules (com "*Ministerio de Educación y Cultura*"), i proporcionar al CRF un marcador explícit de final (E-) li permet aprendre que una entitat no pot continuar un cop tancada, evitant que el model allargui seqüències per error.

Cal destacar que tots els esquemes avançats (BIOW, BIOE, BIOEW) obtenen una Precisió més alta que BIO, però a costa d'un Recall lleugerament inferior, és a dir, el model es torna més conservador i BIOE és el que millor equilibra aquest *trade-off*.

Sense l'etiqueta B- (IO), es perd la capacitat de distingir on acaba una entitat i on en comença una altra del mateix tipus si apareixen consecutivament, cosa que provoca una caiguda notable en el Recall i el situa com la segona pitjor codificació.

Finalment, BIOW+ obté el F1 més baix de tots (0.7633). Assignar etiquetes com O-PER o O-LOC als tokens buits adjacents a una entitat multiplica el nombre de classes possibles sense que hi hagi prou dades per aprendre-les bé, generant dispersió i confusió en el model.

2. Anàlisi per al Neerlandès

Esquema (codificació)	F1-Score	Precisió	Recall
BIO	0.7719	0.7942	0.7508
IO	0.7579	0.7851	0.7325
BIOW	0.7678	0.7905	0.7464
BIOE	0.7669	0.7871	0.7477
BIOEW	0.7695	0.7918	0.7485
BIOW+	0.7624	0.7859	0.7403

Els resultats per al neerlandès, de nou, mostren un patró diferent al de l'espanyol: en aquest cas, cap esquema avançat aconsegueix superar el BIO base, que obté el millor F1 amb 0.7719.

Tots els esquemes alternatius redueixen el F1 respecte al BIO, tant en Precisió com en Recall. Això vol dir que el BIO no només detecta més entitats en general (Recall més alt), sinó que a més ho fa amb més seguretat (Precisió més alta): prediu més entitats i encerta més vegades. Els esquemes avançats, en canvi, no aporten cap benefici clar, ni detecten més entitats ni les classifiquen millor, i el BIO bàsic acaba sent l'opció òptima per al neerlandès en tots els aspectes.

IO torna a ser el segon pitjor esquema (F1 de 0.7579), pel mateix motiu que en espanyol: sense l'etiqueta B-, el model perd la capacitat de separar entitats consecutives del mateix tipus, cosa que penalitza especialment el Recall (0.7325, el més baix de tots els esquemes).

BIOW+ repeteix el comportament observat en espanyol i obté un dels F1 més baixos (0.7624). La multiplicació de classes per als tokens O adjacents genera dispersió en un corpus que ja de per si té menys regularitat ortogràfica que l'espanyol, i el model no té prou dades per aprendre aquestes distincions de forma fiable.

Que el BIO sigui l'esquema òptim per al neerlandès té sentit si ho posem en context amb el que hem vist al *Greedy*: en aquest idioma el model es basa principalment en l'estructura interna de cada paraula (afixos, longitud) i no tant en patrons de seqüència llarga. Afegir marcadors de final d'entitat com E- o W- sembla aportar poc quan la detecció ja es recolza fonamentalment en trets morfològics individuals de cada token.

Rendiment General (Conjunt de Test)

Un cop determinades les configuracions òptimes per a cada idioma (paràmetres i codificació), hem avaluat els models sobre el conjunt de test pur, dades que el sistema no havia vist en cap fase anterior.

Aquest pas és crític per comprovar la capacitat real de generalització dels models. Els resultats obtinguts amb les configuracions guanyadores són els següents:

Idioma	Codificació	F1-Score	Precisió	Recall
Espanyol	BIOE	0.8129	0.8243	0.8018
Neerlandès	BIO	0.7868	0.8076	0.7670

El primer que crida l'atenció és que tots dos models obtenen un F1 superior en test que en validació (0.7889 → 0.8129 en espanyol, 0.7719 → 0.7868 en neerlandès). Això confirma que el *Greedy* ha evitat el sobreajustament de manera efectiva: descartar *features* que no milloraven sobre validació ha resultat en models que generalitzen bé. Cal dir, però, que part d'aquesta millora pot deure's al fet que *testb* (test) i *testa* (validació) són conjunts de dades diferents, i el model pot comportar-se de manera lleugerament diferent segons les frases concretes que contingui cada un, de forma independent a la qualitat del model.

En tots dos idiomes la Precisió és consistentment superior al Recall, cosa que indica que el model és conservador: quan prediu una entitat, encerta amb alta probabilitat, però tendeix a deixar-se'n algunes pel camí. Considerem que en escenaris reals aquest comportament és preferible, ja que minimitza les prediccions errònies.

Finalment, l'espanyol supera el neerlandès per gairebé 3 punts d'F1, una diferència que ja anticipaven els resultats del *Greedy*. L'espanyol es beneficia d'una ortografia més regular i de la disponibilitat de recursos externs (gasetes), mentre que el neerlandès, tot i no comptar amb cap d'aquests avantatges, assoleix un F1 de 0.7868 recolzant-se únicament en patrons morfològics interns.

Prediccions amb text real

Per completar l'avaluació dels models, hem provat els models sobre text cru real, és a dir, frases que no provenen del corpus *CoNLL-2002* i que el model no ha vist mai. A diferència de l'avaluació sobre *testb*, aquí no disposem d'etiquetes de referència, però podem analitzar nosaltres mateixos on encerta i on falla el model.

Anàlisi d'algunes oracions reals en Espanyol

Les primeres oracions analitzades mostren un comportament molt sòlid, amb encerts clars en noms propis freqüents i entitats institucionals comunes. Els errors apareixen principalment en entitats tecnològiques recents, noms artístics atípics i seqüències llargues amb preposicions intercalades.

"El presidente Pedro Sánchez se reunió con la ONU en Nueva York la semana pasada."

Entitat	Predicció	Vàlid	Observació
Pedro Sánchez	PER	Sí	Cap.
ONU	ORG	Sí	Cap.

Nueva York	LOC	Sí	Cap.
------------	-----	----	------

"Mark Petersen está trabajando en Google desde sus oficinas en Barcelona."

Entitat	Predicció	Vàlid	Observació
Mark Petersen	PER	Sí	Cap.
Google	LOC	No	Hauria de ser ORG. El context " <i>trabajando en Google</i> " i els POS-tags (preposició + nom propi) segurament s'associen freqüentment a localitzacions al corpus, i el patró morfològic d'una sola paraula amb majúscula reforça aquesta confusió.
Barcelona	LOC	Si	Correcte, i de fet pel mateix motiu que Google falla: el context " <i>en Barcelona</i> " (preposició + nom propi) és estructuralment idèntic, però en aquest cas la predicció és encertada.

"La Barcelona que por la mañana recibió a Lula da Silva (Brasil) y a Gustavo Petro (Colombia) y que por la noche cantó la llegada de la presidenta de México, Claudia Sheinbaum, la Barcelona del tenis en el Godó y también la de los desfiles en la 080, todo en la misma semana, se ha entregado sin dudarle a Rosalía."

Entitat	Predicció	Vàlid	Observació
Barcelona	LOC	Sí	Cap.
Lula da Silva	LOC	No	Hauria de ser PER. La partícula "da" és una paraula comuna i ambigua que trenca la seqüència de majúscules esperada en un nom de persona, desorientant el model.
Brasil	LOC	Sí	Cap.
Gustavo Petro	PER	Sí	Cap.
Colombia	LOC	Sí	Cap.
México	LOC	Sí	Cap.
Claudia Sheinbaum	PER	Sí	Cap.
Godó	ORG	Sí	La predicció és acceptable però curiosa: amb una finestra de context limitada (2 paraules i 1 POS-tag anteriors), el model probablement ha vist " <i>el</i> " + <i>nom propi</i> i l'ha associat a una institució. Tot i que MISC també seria defensable, el resultat és correcte.
Rosalía	LOC	No	Hauria de ser PER. El context sintàctic " <i>verb + prep + nom propi</i> " és una estructura que ja hem vist abans i que en el corpus s'associa freqüentment a topònims (com ara " <i>viajar a França</i> "). A més, el sufix " <i>-ía</i> " reforça aquesta confusió, ja que coincideix amb el patró morfològic de molts topònims hispànics com Andalucía, Turquía o Galicia, ...

"La llamada a confesarse fue Bad Gyal."

Entitat	Predicció	Vàlid	Observació
---------	-----------	-------	------------

Bad Gyal	No detectat	No	No detectada. Nom artístic en anglès format per paraules comunes i curtes, sense cap patró morfològic que el delati com a entitat.
----------	-------------	----	--

"La alarma la ha encendido Aonan Guan y los investigadores de Johns Hopkins, Zhengyu Liu y Gavin Zhong tras demostrar ataques contra tres agentes desplegados en la mencionada plataforma: Claude Code Security Review, de Anthropic, Gemini CLI Action, de Google, y GitHub Copilot Agent, una herramienta de GitHub bajo Microsoft."

Entitat	Predicció	Vàlid	Observació
Aonan Guan	PER	Sí	Cap.
Johns Hopkins	LOC	No	Hauria de ser ORG. El model interpreta dues paraules capitalitzades consecutives, una d'elles sent <i>Hopkins</i> com una localització: al corpus d'entrenament, seqüències de dos noms propis apareixen freqüentment com a noms de lloc (com " <i>Nueva York</i> ") i " <i>Hopkins</i> " pot ser a gasetes freqüents com una localització.
Zhengyu Liu	PER	Sí	Cap.
Gavin Zhong	PER	Sí	Cap.
Claude / Anthropic / Gemini / Google / GitHub Copilot Agent	PER	No	Haurien de ser etiquetes ORG o MISC. Són entitats tecnològiques "recents" que segurament estan fora del domini del corpus. Sense cap referència prèvia, el model intenta classificar-les per analogia morfològica: paraules com Claude o Gemini segueixen el patró d'un nom propi de persona (una sola paraula amb majúscula inicial), i GitHub Copilot Agent és una seqüència de paraules capitalitzades que el model interpreta com un nom compost de persona.
GitHub	PER	No	Mateix problema que les entitats anteriors: empresa tecnològica fora del domini del corpus, classificada com a PER per analogia morfològica amb noms propis de persona.
Microsoft	ORG	Sí	A diferència de les anteriors, <i>Microsoft</i> al ser una organització <i>antiga</i> pot ser prou present al corpus d'entrenament com per haver après la seva categoria correcta.

"Veremos cómo resuelve este desafío Dishan Technology. Por otro lado, Moore Threads ha desarrollado varias GPU para aplicaciones de IA que, sobre el papel, rivalizan con algunas de las soluciones avanzadas que han colocado en el mercado NVIDIA, AMD o Huawei."

Entitat	Predicció	Vàlid	Observació
Dishan Technology	PER	No	Hauria de ser ORG. El model interpreta la seqüència de dues paraules capitalitzades com un nom de persona, un patró molt freqüent al corpus d'entrenament. A més, tant " <i>Dishan</i> " com " <i>Technology</i> " són paraules angleses que el model no reconeix, i els POS-tags que <i>Stanza</i> els assigna dins

			de text en espanyol poden ser incorrectes, eliminant una de les features més útils per distingir entre tipus d'entitat.
Moore Threads	PER	No	Mateix problema: dues paraules capitalitzades en anglès sense context suficient. "Moore" segueix el patró morfològic d'un cognom anglosaxó, cosa que reforça la classificació errònia com a PER, i "Threads" és una paraula anglesa que el model no associa amb cap categoria clara en el context del corpus espanyol.
IA / NVIDIA / AMD / Huawei	ORG	Sí	Correcte per a la majoria (tot i que IA és discutible). NVIDIA, AMD i Huawei són empreses prou antigues i consolidades com per haver aparegut al corpus d'entrenament del 2002, i a més segueixen el patró de sigles o noms en majúscules que el model associa clarament a organitzacions.

Anàlisi d'algunes oracions reals en Neerlandès

"Verstappen was op het moment van de crash niet op het circuit."

(Verstappen no era a la pista en el moment de l'accident.)

Entitat	Predicció	Vàlid	Observació
Verstappen	No detectat	No	El model no disposa de cap pista morfològica clara: és un cognom sol, sense nom de pila que reforci el patró de persona ni cap títol o càrrec previ ("el pilot", "el corredor") que actui de context. A diferència dels noms de persona complets (com Lucas Auer a la següent oració, on el nom de pila és una pista clara), Verstappen és una paraula única que morfològicament no segueix cap dels patrons de sufix típics del neerlandès associats a noms propis, i el model no té prou evidència per classificar-la com a entitat.

"Zijn teamgenoot Lucas Auer was begonnen aan de race, die tot 21.30 uur zou duren."

(El seu company d'equip Lucas Auer havia començat la cursa, que duraria fins a les 21:30.)

Entitat	Predicció	Vàlid	Observació
Lucas Auer	PER	Sí	Correcte. A diferència de Verstappen a l'oració anterior, aquí el model disposa de dues paraules capitalitzades consecutives on la primera ("Lucas") és un nom de pila que pot ser molt freqüent al corpus, cosa que proporciona prou context morfològic per identificar correctament la seqüència com a PER.

"De gouverneur van New Jersey, Mikie Sherrill, heeft de internationale voetbalbond FIFA opgeroepen de transportkosten te vergoeden."

(La governadora de Nova Jersey, Mikie Sherrill, ha demanat a la federació internacional de futbol FIFA que reemborsi les despeses de transport.)

Entitat	Predicció	Vàlid	Observació
New Jersey	PER	No	Hauria de ser LOC. El problema és doble: d'una banda, és un topònim anglès format per dues paraules comunes ("New" i "Jersey"), sense cap dels sufixos típics neerlandesos que el model associa a localitzacions (com -dam, -burg o -straat). De l'altra, el context immediat "van New Jersey" (preposició + nom propi) podria haver estat útil, però el model neerlandès utilitza una finestra de context de només 1 paraula, i "van" sol aparèixer també davant de noms de persona en neerlandès (com en "van Gogh"), cosa que probablement ha desviat la classificació cap a PER.
Mikie Sherrill	PER	Sí	Cap.
FIFA	ORG	Sí	Correcte. És una sigla formada per quatre majúscules, un patró que el model associa clarament a organitzacions, i a més és una entitat que pot estar prou present al corpus d'entrenament per haver estat correctament apresada.

"Oprichter Dario Amodèi van AI-bedrijf Anthropic heeft op het Witte Huis de ontwikkeling van super-AI Mythos besproken."

(Dario Amodèi, fundador de l'empresa d'IA Anthropic, va parlar sobre el desenvolupament de la super-IA Mythos a la Casa Blanca.)

Entitat	Predicció	Vàlid	Observació
Dario Amodèi	PER	Sí	Cap.
AI-bedrijf Anthropic	MISC	No	Hauria de ser ORG per a "Anthropic". Probablement el model ha fusionat el substantiu comú "AI-bedrijf" amb el nom propi "Anthropic", tractant-los com una única entitat composta. El fet que "AI-bedrijf" contingui un guió i lletres majúscules (AI) pot haver contribuït a la classificació com a MISC, ja que aquest tipus de compostos híbrids no encaixen clarament en cap de les categories principals i el model els tracta com a entitats de naturalesa indefinida. A més, segurament "Anthropic" és una empresa molt recent que difícilment apareix al corpus d'entrenament.
Witte Huis	LOC	Pot Ser	Com a edifici físic LOC és defensable, però com a seu del poder executiu seria més precís ORG. El model probablement l'ha classificat com a LOC perquè "Witte" és un adjectiu i "Huis" ("casa") és un nom comú que morfològicament s'assembla a un lloc físic, i aquesta combinació (adjectiu + nom de lloc) és un patró habitual per a localitzacions en neerlandès.
super-AI Mythos	PER	No	Hauria de ser MISC o ORG. Mythos segueix el patró morfològic d'un nom propi de persona, pel que el model interpreta tota la seqüència com un nom de persona en lloc del nom d'un sistema d'IA.

"De RDW heeft goedkeuring gegeven voor het rijhulpsysteem Full Self Driving (Supervised)."

(L'RDW ha atorgat l'aprovació al sistema d'assistència al conductor de conducció totalment autònoma (supervisada).)

Entitat	Predicció	Vàlid	Observació
RDW	ORG	Sí	Correcte. És un acrònim de tres majúscules, un patró que el model associa clarament a organitzacions, i a més és l'acrònim del Rijkdienst voor het Wegverkeer, l'agència governamental neerlandesa de trànsit, que probablement apareix al corpus d'entrenament neerlandès.
Full Self Driving	PER	No	Hauria de ser MISC. És terminologia tècnica anglesa formada per tres paraules capitalitzades consecutives, un patró que el model neerlandès associa freqüentment a noms de persona. A més, segurament el model no ha vist mai aquesta expressió al corpus d'entrenament, i sense cap sufix o context morfològic que indiqui que es tracta d'un sistema tecnològic, aplica el patró més habitual per a seqüències d'inici de majúscules: nom de persona.
Supervised	LOC	No	Hauria d'ignorar-se o classificar-se com a MISC. Apareix entre parèntesis de manera aïllada, sense cap context adjacent que reforci cap categoria concreta. El model probablement la classifica com a LOC perquè una paraula capitalitzada sola, sense nom de pila previ ni cognom posterior, no encaixa prou bé amb el patró de persona, i la terminació "-ed" podria assemblar-se morfològicament a algunes localitzacions del corpus.

Tipologia d'errors

Analitzant conjuntament les prediccions dels dos idiomes, podem identificar les següents categories d'error:

1. Confusió de Categoria (error de tipus, no de detecció)

És l'error més freqüent. El model detecta que hi ha una entitat però li assigna la categoria incorrecta. Els casos més habituals són ORG classificada com a LOC o PER (*Google* → LOC, *Dishan Technology* → PER, etc.). La causa principal és segurament que el CRF no té cap coneixement semàntic de les entitats: no sap què és *Google* ni què és *Dishan Technology*. En el seu lloc, pren decisions basades exclusivament en patrons morfològics i de context. El problema és que moltes entitats de categories diferents comparteixen exactament la mateixa forma visual (una o dues paraules amb majúscula inicial), de manera que el model no pot distingir-les per morfologia i acaba aplicant el patró estadísticament més freqüent al corpus per a aquella estructura concreta. Si al corpus d'entrenament les paraules amb aquella forma i en contextos similars apareixien majoritàriament com a persones o llocs, el model les classificarà com a tals independentment del seu significat real.

2. Entitats fora del Domini del Corpus i Terminologia Anglesa

El corpus *CoNLL-2002* és del 2002, per tant, conté entitats del món d'aquella època. Noms com *Anthropic*, *ChatGPT*, *OpenAI* i *Gemini* no tenien cap representació en les dades d'entrenament, i el model intenta classificar-les per analogia morfològica amb resultats

inconsistents. A més, la majoria d'aquestes entitats són termes en anglès (com altres errors com *Full Self Driving*, *Supervised* i *Dishan Technology*), un fenomen cada vegada més freqüent degut a la creixent adopció d'anglicismes, especialment en l'àmbit tecnològic. Això agreuja el problema: no només són entitats noves i desconegudes per al model, sinó que a més segueixen patrons morfològics diferents a l'espanyol i al neerlandès, cosa que fa que els POS-tags generats per *Stanza* puguin ser incorrectes i que les *features* morfològiques perdin bona part del seu valor discriminatiu.

3. Entitats Multiparaula amb Paraules Ambigües

Les entitats formades per múltiples paraules on alguna d'elles és una paraula comuna (preposicions, verbs, articles) plantegen un repte important per als models. El CRF aprèn a identificar entitats basant-se en seqüències de tokens amb certes característiques morfològiques, i quan aquesta seqüència es veu interrompuda per una paraula amb un POS-tag de verb, preposició o article, el patró esperat es trenca i el model perd el fil de l'entitat. *Lula da Silva* és el cas més clar: la partícula *da* és una paraula ambigua que pot interpretar-se com a verb o preposició, i el seu POS-tag trenca la cadena de majúscules esperada en un nom de persona. En neerlandès, *New Jersey* falla pel mateix motiu: *New* és una paraula anglesa comuna que el model no reconeix com a part d'una localització sinó com adjectiu, i el context sintàctic "*van New Jersey*" (preposició + adjectiu + nom) no és prou discriminatiu per revelar que es tracta d'una localització composta.

4. Noms artístics, Pseudònims i Entitats Atípiques

Els noms artístics i les organitzacions polítiques o paramilitars presenten una dificultat especial per al model, ja que sovint no segueixen cap dels patrons morfològics convencionals associats a les categories estàndard del NER. *Bad Gyal* és un exemple clar: és un nom artístic format per dues paraules angleses comunes a les que *Stanza* probablement els assigna POS-tags incorrectes en ser dins de text en espanyol, eliminant una de les *features* més rellevants del model i fent pràcticament impossible la detecció. El cas de *Hezbollah* en neerlandès és diferent: el model sí que la detecta com a entitat, però la classifica com a PER en lloc d'ORG. La raó probable és que morfològicament s'assembla a un nom propi de persona (una sola paraula capitalitzada sense cap sufix típic d'organització) però que en absència de context que indiqui clarament que es tracta d'un moviment polític, el model aplica el patró estadísticament més freqüent per a paraules amb aquesta forma.

5. Inconsistència entre Oracions

El CRF no té memòria entre frases: cada oració s'avalua de manera completament independent, sense cap noció del que ha predit abans. Això provoca que la mateixa entitat rebi categories diferents segons el context local de cada frase, ja que la decisió es pren únicament a partir dels tokens adjacents i els POS-tags de la finestra de context immediata (la resta d'etiquetes no canvien). El cas més clar és OpenAI en neerlandès: LOC a l'oració 16 i PER a la 17, simplement perquè el context sintàctic que l'envolta és diferent en cada cas. En espanyol, Google apareix com a LOC a l'oració 1 i com a PER a l'oració 10, pel mateix motiu. Aquesta limitació és estructural i inherent al disseny del CRF: sense memòria global ni cap mecanisme per mantenir la coherència entre prediccions de frases diferents, el model no pot garantir que una mateixa entitat rebi sempre la mateixa etiqueta al llarg d'un text.

Anotació 1: Cal aclarir que la numeració de les oracions (com oració 16) es correspon amb la de la sortida del notebook, i que les oracions analitzades aquí són només **un subconjunt** del total processat.

Anotació 2: Les oracions en espanyol s'han extret de mitjans com *La Vanguardia*, *El País*, *Xataka* i *BBC News*, mentre que les oracions en neerlandès provenen dels mitjans de comunicació neerlandesos *NU.nl* i *NOS.nl*.

Dificultats

Durant el desenvolupament del projecte ens vam trobar amb dues dificultats principals que van condicionar de manera significativa l'evolució del treball.

La primera va ser el cost computacional de l'aproximació inicial. En un primer moment vam plantejar un *Grid Search* manual que provava totes les combinacions possibles de característiques. L'execució trigava moltes hores i en cap cas arribava a acabar en un temps raonable, cosa que feia inviable qualsevol experimentació. Això ens va portar a replantejar l'estratègia i adoptar la cerca *Greedy*, que construeix la configuració òptima de manera incremental i redueix el nombre d'entrenaments necessaris sense sacrificar la qualitat dels resultats.

La segona dificultat va ser la normalització dels POS-tags de *Stanza* per fer-los compatibles amb el corpus *CoNLL-2002*. Com s'ha explicat, el corpus utilitza codificacions diferents per a cada idioma i *Stanza* genera les seves etiquetes en formats que no coincideixen directament amb cap dels dos. Determinar les regles de transformació correctes va requerir molts *prints* comparatius, analitzant token a token les diferències entre les etiquetes del corpus i les generades per *Stanza* sobre les mateixes oracions, fins a trobar un mapeig bastant adequat per a cada cas.

Conclusions

Hem completat tots els objectius proposats al projecte, dissenyant i avaluant un sistema de Reconeixement d'Entitats Anomenades basat en CRF per a l'espanyol i el neerlandès amb un rendiment prou sòlid.

La cerca *Greedy* ha demostrat ser una estratègia eficient per seleccionar les característiques més rellevants, i els resultats obtinguts reflecteixen de manera clara les diferències morfològiques entre els dos idiomes.

Pel que fa als esquemes d'etiquetatge, hem comprovat que una codificació més rica no sempre és millor: BIOE és l'opció òptima per a l'espanyol mentre que el neerlandès funciona millor amb BIO estàndard.

L'avaluació final sobre el conjunt de test ha confirmat que els models generalitzen bé, amb un F1 de 0.8129 per a l'espanyol i 0.7868 per al neerlandès. Les prediccions sobre text cru han mostrat els límits inherents dels models, principalment la dependència del domini del corpus d'entrenament i la dificultat amb entitats recents, atípiques o en llengües que el model no ha vist mai.